

RECOVERY TECHNIQUES IN DATABASES

Recovery management in DBMS is used to restore the database to a consistent state after failures such as system crash, power failure, software error, or transaction failure.

The main objective of recovery is to ensure the ACID properties of transactions, especially:

- Atomicity
- Durability

A recovery system uses a log file that stores all transaction activities.

Log-Based Recovery

- Log-Based Recovery is a recovery mechanism in which the DBMS maintains a log file containing all transaction activities so that the database can be restored to a consistent state after a crash or failure.
- The log file is stored on stable storage.

What is stored in Log File?

The log generally contains:

Log Record	Meaning
<START T1>	Transaction T1 started
<T1, X, 100, 150>	T1 changed X from 100 to 150
<COMMIT T1>	T1 committed
<ABORT T1>	T1 aborted

Purpose of Log-Based Recovery

The log helps DBMS to:

1. Undo incomplete transactions
2. Redo committed transactions

3. Maintain consistency
4. Ensure atomicity and durability

Types of Log-Based Recovery

1. Deferred Update Recovery Technique

In Deferred Update:

- Changes made by a transaction are not immediately written to the actual database.
- Updates are first stored in the log or buffer.
- The database is updated only after the transaction commits.

This technique is also called: **No-UNDO/REDO technique**

because:

- Uncommitted transactions never update the database, so UNDO is not needed.
- Committed transactions may need REDO after a crash.

Working of Deferred Update

1. Transaction starts.
2. All changes are recorded in log.
3. Actual database is not modified immediately.
4. If transaction commits:
 - Updates are written to database.
5. If crash occurs before commit:
 - Ignore transaction completely.

Example

Suppose:

Initial value: $X = 100$

Transaction T1: $X = 150$

Log:

Log Records
<START T1>
<T1, X, 100, 150>
<COMMIT T1>

If crash occurs after commit but before writing data permanently to database:

Recovery system performs: REDO(T1)

Final value: $X = 150$

Characteristics of Deferred Update

Feature	Description
Database updated before commit	No
Need of UNDO	No
Need of REDO	Yes
Simpler recovery	Yes
Risk of inconsistency	Low

Advantages

1. Simple recovery process.
2. No need to undo incomplete transactions.
3. Maintains atomicity easily.

Disadvantages

1. Changes are delayed.
2. Slower commit operation.

3. Less concurrency in some cases.

2. Immediate Update Recovery Technique

In Immediate Update:

- Database can be updated before transaction commits.
- Changes are written immediately to database.
- Therefore, if crash occurs:
 - Committed transactions may need REDO.
 - Uncommitted transactions must be UNDONE.

This technique is also called: **UNDO/REDO technique**

Working of Immediate Update

1. Transaction starts.
2. Updates are written immediately to database.
3. Before updating database:
 - Old value and new value are stored in log.
4. If transaction commits:
 - Changes remain permanent.
5. If crash occurs:
 - Undo incomplete transactions.
 - Redo committed transactions if needed.

Example

Initial values: A = 500

Transaction T1 updates: A = 700

Log:

Log Records
<START T1>
<T1, A, 500, 700>

Suppose crash occurs before commit.

Since database may already contain: A = 700

Recovery system performs: UNDO(T1)

Restored value: A = 500

Characteristics of Immediate Update

Feature	Description
Database updated before commit	Yes
Need of UNDO	Yes
Need of REDO	Yes
Faster updates	Yes
Complex recovery	Yes

Advantages

1. Faster database updates.
2. Better concurrency.
3. Efficient for large systems.

Disadvantages

1. Recovery process is complex.
2. Requires both UNDO and REDO.
3. More logging overhead.

3. Checkpoint-Based Recovery

- Checkpoint is a recovery optimization technique.
- Instead of scanning the entire log after crash, DBMS creates checkpoints periodically.

Checkpoint stores:

- List of active transactions
- Current status of database buffers

After crash:

- Recovery starts from latest checkpoint instead of beginning of log.

Working of Checkpoint

1. Temporarily stop transaction processing.
2. Flush all log records to stable storage.
3. Flush modified database buffers to disk.
4. Write <CHECKPOINT> record in log.
5. Resume transaction execution.

Example

Log:

Log Records
<START T1>
<T1, A, 1000, 1200>
<START T2>
<CHECKPOINT>
<T2, B, 500, 800>
<COMMIT T2>
SYSTEM CRASH

After crash:

- Recovery begins from checkpoint.
- T2 committed → REDO
- T1 incomplete → UNDO

Characteristics of Checkpoint Recovery

Feature	Description
Reduces recovery time	Yes
Uses checkpoints in log	Yes
Scans full log	No
Improves efficiency	Yes

Advantages

1. Faster crash recovery.
2. Reduces log scanning time.
3. Improves system performance.

Disadvantages

1. Creating checkpoint adds overhead.
2. Temporary pause may occur.

Write-Ahead Logging (WAL)

Write-Ahead Logging means:

Before any database item is updated on disk, its log record must first be written to stable storage.

WAL Rules

1. Log record must be written before database update.

2. Commit record must be written before transaction completes.

Importance of WAL

1. Enables recovery after crash.
2. Supports REDO and UNDO operations.
3. Prevents data inconsistency.
4. Ensures durability.

Comparison of Recovery Techniques

Feature	Deferred Update	Immediate Update	Checkpoint Recovery
Database updated before commit	No	Yes	Depends
UNDO needed	No	Yes	Yes
REDO needed	Yes	Yes	Yes
Recovery speed	Moderate	Slower	Faster
Complexity	Simple	Complex	Moderate
Uses checkpoint	No	No	Yes

PRACTICE QUESTIONS: LOG BASED RECOVERY

Question 1

A DBMS uses immediate update technique and maintains the following log records:

Log Records
<START T1>
<T1, A, 500, 700>
<START T2>
<T2, B, 300, 450>
<COMMIT T1>
<T2, C, 200, 350>
SYSTEM CRASH

The initial values are: A = 500, B = 300, C = 200

Determine the recovery actions performed after the crash and find the final values of A, B, and C.

Solution

Step 1: Identify committed and uncommitted transactions

- T1 has committed.
- T2 has not committed.

Therefore:

- REDO(T1)
- UNDO(T2)

Step 2: REDO committed transaction

T1 updated:

- A : 500 $\rightarrow$ 700

After REDO: A = 700

Step 3: UNDO incomplete transaction

T2 updated:

- B : 300 $\rightarrow$ 450
- C : 200 $\rightarrow$ 350

Undo restores old values:

- B = 300
- C = 200

Final Values After Recovery

A:700, B:300, C:200

Question 2

A database system follows deferred update technique and maintains the following log:

Log Records
<START T1>
<T1, X, 100, 150>
<START T2>
<T2, Y, 200, 250>
<COMMIT T1>
SYSTEM CRASH

Assume initial values are: X = 100, Y = 200

Explain the recovery process and determine the final database state after crash recovery.

Solution

Step 1: In deferred update:

- No UNDO required.
- Only committed transactions are REDONE.

Step 2: Identify committed transactions

- T1 committed.
- T2 did not commit.

Therefore:

- REDO(T1)
- Ignore T2

Step 3: Apply REDO

T1 changed: X : 100 → 150

T2 is ignored completely because it never committed.

Final Values After Recovery

X: 150, Y:200

Question 3

Consider the following log maintained using checkpoint-based recovery:

Log Records
<START T1>
<T1, A, 1000, 1200>
<START T2>
<T2, B, 500, 800>
<CHECKPOINT>
<T1, C, 700, 900>
<COMMIT T2>

SYSTEM CRASH

The initial values are: $A = 1000$, $B = 500$, $C = 700$

Using checkpoint recovery, determine the transactions that require REDO and UNDO operations. Also find the final values of A, B, and C.

Solution

Step 1: Identify transaction status

- T2 committed before crash.
- T1 did not commit.

Therefore:

- REDO(T2)
- UNDO(T1)

Step 2: Start recovery from checkpoint

Checkpoint reduces recovery work because scanning starts from latest checkpoint.

Transactions active during checkpoint:

- T1
- T2

Step 3: Apply REDO

T2 updated:

- $B : 500 \rightarrow 800$

After REDO:

- $B = 800$

Step 4: Apply UNDO

T1 updates:

- $A : 1200 \rightarrow 1000$

- C : 900 $\rightarrow$ 700

Restore original values.

Final Values After Recovery

A:1000, B:800, C:700

GUNJAN CHUGH